

Terraform

Flow of terraform

1. Introduction to IAC
2. Why do we need IAC?
3. Tools of IAC
4. What is terraform
5. History of terraform
6. Features of terraform
7. Architecture of terraform
8. Lifecycle of terraform
9. Terraform hands on practice
 - Installation of terraform
 - Creation of EC2, S3, VPC in terraform
 - Variables in terraform
 - Terraform output
 - Modules in terraform

1. Introduction to IAC

IAC - Infrastructure as code

>> It is the process of creating and managing IT infrastructure using the code instead of manually doing it.

Infrastructure such as - Servers : EC2

- Database : RDS

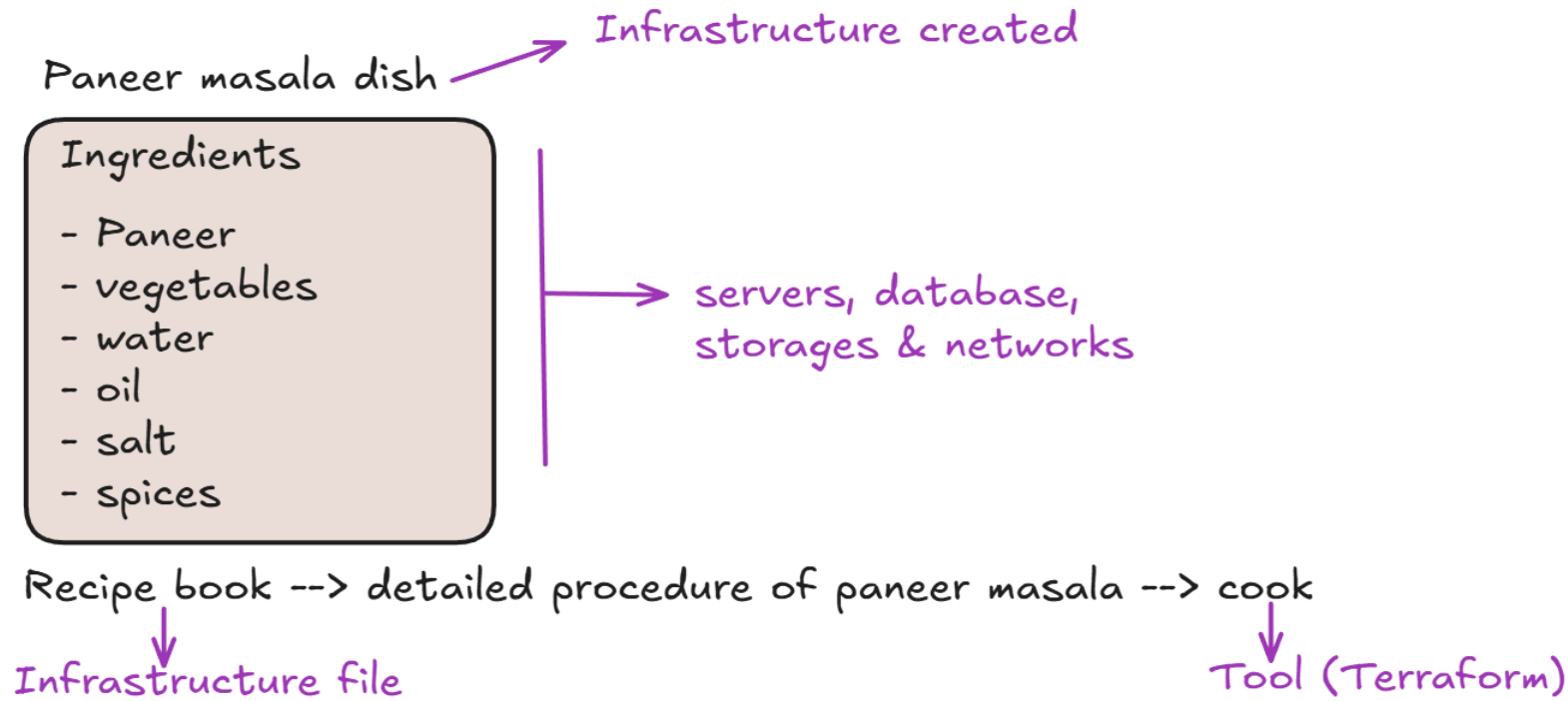
- Networks : VPC

- Storage : S3, EBS

>> Here, the code will be written in order to create the resources(Servers, database, networks, storage) automatically.

>> With infrastructure as code, it enables to represent the infrastructure through machine readable configuration files that supports automation, sharing file across team and version control.

Ex :



2. Why do we need IAC?

1. Scalability - You can easily increase or decrease the resources of infrastructure depending on requirement.
2. Automation - You can easily update or delete any resources or modify the infrastructure file whenever needed using commands.
3. Repeatability - Same infrastructure can be created using the same code.
4. Faster process - Provisioning can happen very quickly.
5. Version control - Tracking the changes made in infrastructure file.
6. Security - Providing the security rules in infrastructure file.
7. Consistency - To ensure same infrastructure file is created every time we run the code due to which we can avoid human errors too.

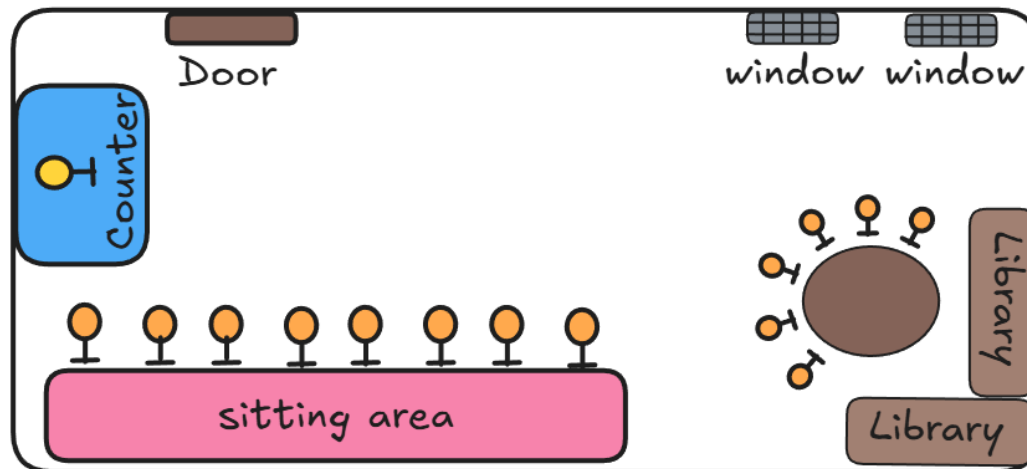
Tools of IAC

1. Terraform
2. Puppet
3. Pulumi
4. Ansible
5. Bicep
6. AWS CloudFormation

Introduction to terraform

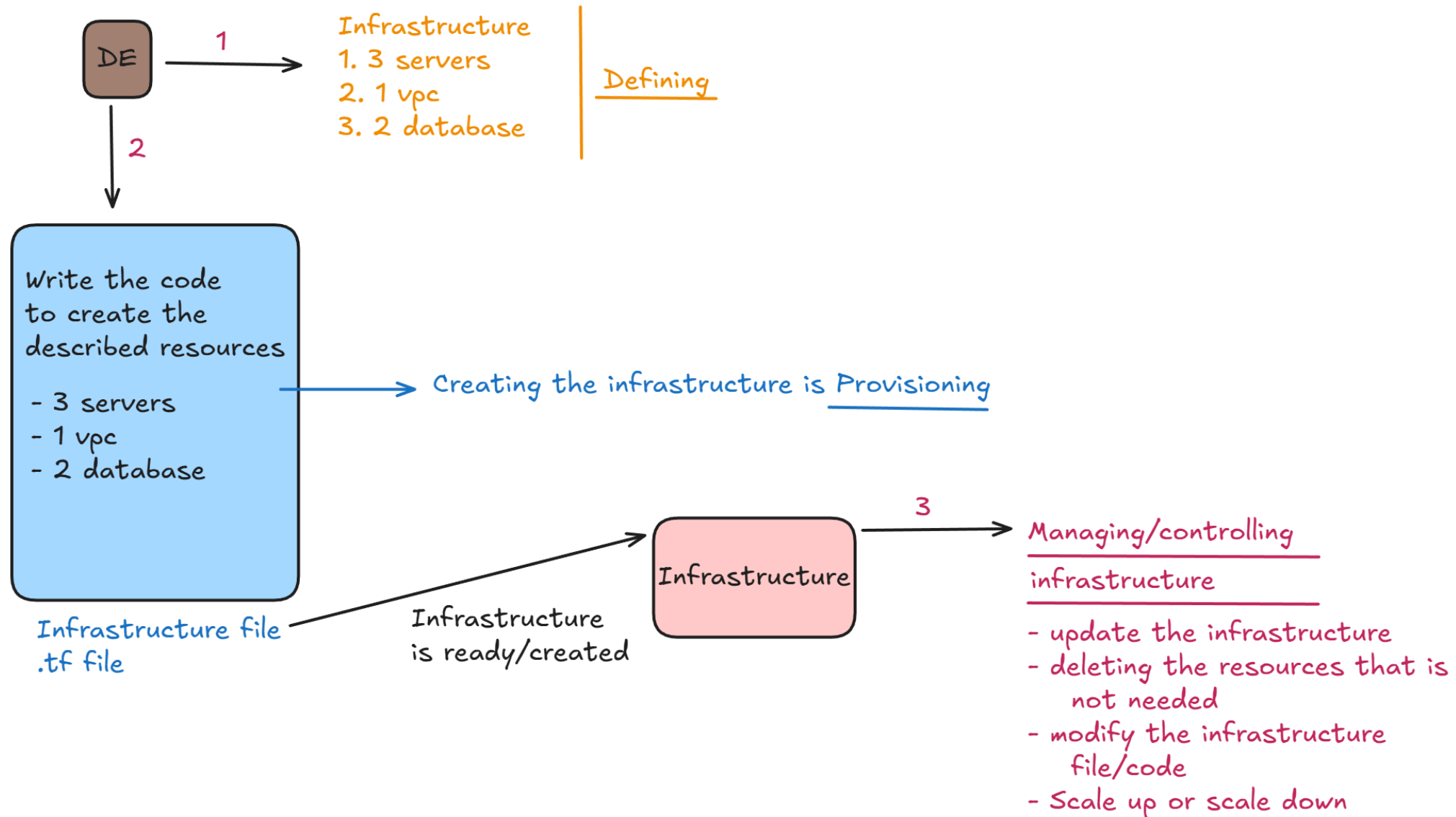
- >> Terraform is an open source infrastructure as code tool created by HashiCorp that can be used freely to manage infrastructure.
- >> Instead of manually performing the required task terraform will help to create the resources like servers, databases, storages and networks automatically.
- >> Terraform enables to describe, create & control the infrastructures in different cloud platform.
 - It lets you define, provision & manage the infrastructure that is been created.

Ex : **Cafe coffee**



--> The blueprint of cafe

Ex :



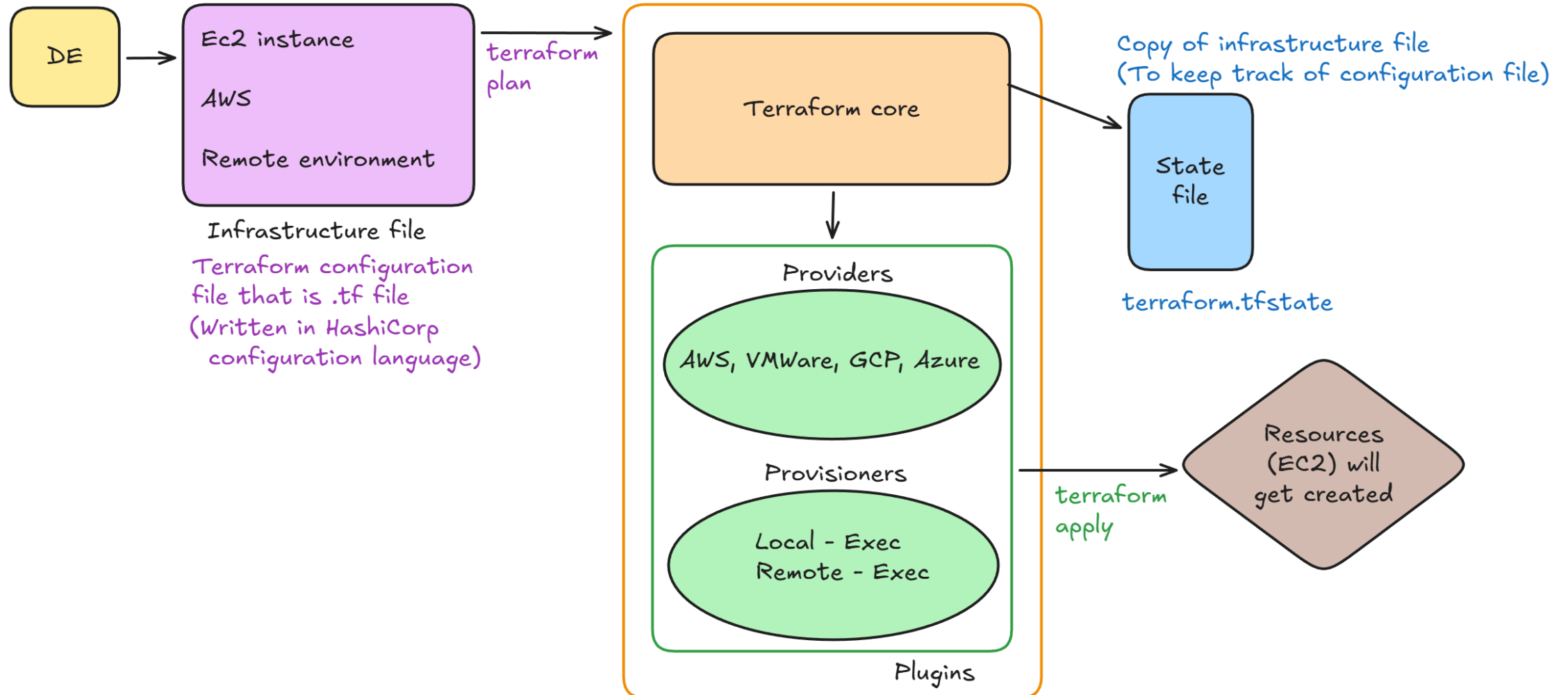
History of terraform

- >> Terraform is one of the major tool created by HashiCorp company that was founded by Mitchell Hashimoto and Armon Dadgar in the year 2012.
- >> It was officially released in the year 2014 as an open source IAC tool.

Features of terraform

1. It is free and open source
2. Terraform uses declarative approach (JSON format or HCL)
3. It helps tracking & managing the infrastructure state in a file.
4. Terraform shows what will change before execution
5. Terraform works with multiple cloud providers(AWS, GCP, Azure, VMWare.....)

Architecture of terraform



Lifecycle of terraform

>> It defines the sequence of steps that terraform has to follow to create, update, setup configurations, manage the resources and delete the resources if it is not needed.

4 phases of terraform lifecycle.

1. Initialization
2. Planning
3. Apply
4. Destroy

1. Initialization

>> It is the first step in terraform lifecycle where in terraform will prepare the directory by downloading all the necessary components.

- downloading the required providers
- install required plugins
- prepare the terraform to execute any commands smoothly.

command : terraform init

2. Planning

>> It will generate execution preview that shows what terraform intends to change before making the actual execution.

command : terraform plan

3. Apply

>> It is used to execute the changes proposed in the plan & accordingly it will create or update the resources.

command : terraform apply

4. Destroy

>> It is used to delete the resources managed by terraform as described in the infrastructure file.

command : terraform destroy

Step-by-Step Terraform Workflow

1. To Connect

- Go to AWS console and select IAM service then create a user.
- Select the created user then go to security credentials section & create access key.
- Open command prompt as administrator & pass the command as aws configure
- Provide access key, secret access key, region name & output format.

2. Folder Setup

- Create a main folder then create a subfolder inside it & open the subfolder in VS Code

3. Create Terraform Files

- Create two files:
 - providers.tf : used to download cloud provider.
 - main.tf : used to write infrastructure code.

4. Configure Provider (providers.tf)

- Go to terraform registry & select browse providers then click on AWS
- Click use provider & copy the AWS provider code and paste it into providers.tf
- Update the region & run the command terraform init (downloads provider and initializes terraform)

5. Create EC2 Instance (main.tf)

- Go back to terraform registry & from AWS provider open documentation & search for EC2
- Under resources select aws_instance & copy the EC2 resource code and paste it into main.tf
- Update : Resource alias name, AMI ID (from AWS EC2 → AMI catalog), instance type, Instance name (tags).

6. Create and Attach Key Pair

- In terraform registry, search for AWS key pair & copy the key pair resource code and paste it into main.tf
- Generate public key :
 - Run ssh-keygen in terminal, go to .ssh directory, locate the .pub file & use cat <filename>.pub to copy the public key
- Paste the public key in the key pair resource & reference the key pair name in the EC2 instance block so it attaches during creation.

7. Execute Terraform Commands

- Run the command terraform plan to preview changes to terraform & then terraform apply to create infrastructure
- Verify resources : Check AWS Console or use terraform show to view created infrastructure.

8. Clean Up Resources

- If resources are no longer needed then run terraform destroy to delete EC2 instance, key pair and related resources.

providers.tf file of EC2

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "6.27.0"  
    }  
  }  
}
```

```
provider "aws" {
  region = "us-west-2"
}
```

```
# to create the infrastructure
# to create instance
resource "aws_instance" "instancealias" {
  ami           = "ami-055a9df0c8c9f681c" # Amazon linux OS image ID
  instance_type = "t3.micro"
  key_name = aws_key_pair.keypairalias.key_name #syntax --> resourcename.aliasname.id

  tags = {
    Name = "terrainstance"
  }
}

# to create keypair
resource "aws_key_pair" "keypairalias" {
  key_name = "terrakeypair"
  public_key = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIL75AoFYjtAd
  ZPB/i6JmjknFN/a9hd142zdMDlpeEVUktanvi@DESKTOP-7EKTOLQ"
}

# to create ebs volume
resource "aws_ebs_volume" "ebsalias" {
  availability_zone = "us-west-2b"
  size              = 45
  tags = {
    Name = "terravolume"
  }
}

# to attach the volume to instance
resource "aws_volume_attachment" "attachalias" {
  device_name = "/dev/xvdbf"
  volume_id   = aws_ebs_volume.ebsalias.id # resourcename.aliasname.id --> syntax of volume_id
  instance_id = aws_instance.instancealias.id # resourcename.aliasname.id --> syntax of instance_id
}
```


providers.tf file of S3

```
# to specify the cloud provider & region to create resources
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "6.27.0"
    }
  }
}

provider "aws" {
  region = "us-west-2"
}
```

main.tf file of S3

```
# to create s3 bucket
resource "aws_s3_bucket" "bucketalias" {
  bucket = "terrabucket78901"

  tags = {
    Name      = "My bucket"
  }
}

# to stop blocking the public access
resource "aws_s3_bucket_public_access_block" "blockaccessalias" {
  bucket = aws_s3_bucket.bucketalias.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls     = false
  restrict_public_buckets = false
}

# to enable bucket versioning
resource "aws_s3_bucket_versioning" "versionalias" {
  bucket = aws_s3_bucket.bucketalias.id
  versioning_configuration {
    status = "Enabled"
  }
}

# to add object in bucket
resource "aws_s3_object" "objectalias" {
  bucket = "terrabucket78901"
  key    = "terraobject"
  source = "C:/Users/Tanvi/Downloads/m3ekskey.pem"
}
```

providers.tf file of VPC

to specify the cloud provider & region to create resources

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "6.27.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "us-west-2"  
}
```

main.tf file of VPC

to create vpc

```
resource "aws_vpc" "vpcalias" {  
  cidr_block      = "10.0.0.0/24" # to get 256 IP's we go for 10.0.0.0/24  
  instance_tenancy = "default"
```

```
  tags = {  
    Name = "terravpc"  
  }  
}
```

```

# to create public subnet
resource "aws_subnet" "publicsubnetalias" {
  vpc_id      = aws_vpc.vpcalias.id
  cidr_block = "10.0.0.0/25"

  tags = {
    Name = "publicsubnet"
  }
}

# to create private subnet
resource "aws_subnet" "privatesubnetalias" {
  vpc_id      = aws_vpc.vpcalias.id
  cidr_block = "10.0.0.128/25"

  tags = {
    Name = "privatesubnet"
  }
}

# to create public route table
resource "aws_route_table" "publicrtalias" {
  vpc_id = aws_vpc.vpcalias.id

  route {
    cidr_block = "10.0.1.0/24"
    gateway_id = aws_internet_gateway.igwalias.id
  }

  tags = {
    Name = "public_rt"
  }
}

```

```

# to create private route table
resource "aws_route_table" "privatertalias" {
  vpc_id = aws_vpc.vpcalias.id

  tags = {
    Name = "private_rt"
  }
}

# to create internet gateway
resource "aws_internet_gateway" "igwalias" {
  vpc_id = aws_vpc.vpcalias.id

  tags = {
    Name = "terraigw"
  }
}

# to attach public route table to public subnet
resource "aws_route_table_association" "alias1" {
  subnet_id      = aws_subnet.publicsubnetalias.id
  route_table_id = aws_route_table.publicrtalias.id
}

# to attach private route table to private subnet
resource "aws_route_table_association" "alias2" {
  subnet_id      = aws_subnet.privatesubnetalias.id
  route_table_id = aws_route_table.privatertalias.id
}

```

Terraform variables

>> It is used to store values that can be changed & it makes the infrastructure file or terraform code flexible & reusable.

Uses of variables

1. To keep the code secure
2. To change values without touching the main file(main.tf)
3. To reuse same code across different environment
4. To make the code clean.

Types of variables in terraform

1. String - text
2. Map - key value pair
3. Number - numeric value
4. List - n no of values or multiple values
5. Bool - true or false

Syntax :

```
variable "resource_name" {  
    description = "detail of what the variable is for" #optional  
    type        = number                             #optional  
    default     = "value"  
}
```

To use variable values in main file :

>> var.resource_name

List of variables in EC2, S3 & VPC

EC2

ami_id, key_name, instance_type, instance_name, security_group_ids
user_data, availability_zone, associate_public_ip, root_volume_size, subnet_id

S3

bucket_name, environment, tags, acl, encryption, object_name, enable_versioning, force_destroy

VPC

vpc_name, subnet_name, route_table_name, vpc_cidr, public_subnet_cidrs, private_subnet_cidrs,
availability_zone, enable_internet_gateway, enable_nat_gateway, enable_dns_hostnames

main.tf file of EC2

```
# to create the infrastructure
# to create instance
resource "aws_instance" "instancealias" {
  ami           = "ami-055a9df0c8c9f681c" # Amazon linux OS image ID
  instance_type = var.instance_type
  key_name      = aws_key_pair.keypairalias.key_name #syntax --> resourcename.aliasname.id
  tags = {
    Name = "terrainstance" #instance name
  }
}

# to create keypair
resource "aws_key_pair" "keypairalias" {
  key_name   = var.key_name
  public_key = "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIL75AoFYjtAdZPB/i6JmjknFN/a9h
d142zdMDlpeEVUk tanvi@DESKTOP-7EKTOLQ"
}

# to create the volume
resource "aws_ebs_volume" "volumealias" {
  availability_zone = var.availability_zone
  size              = var.size

  tags = {
    Name = "terravolume"
  }
}

# to attach volume to instance
resource "aws_volume_attachment" "ebs_att" {
  device_name = var.device_name
  volume_id   = aws_ebs_volume.volumealias.id
  instance_id = aws_instance.instancealias.id
}
```

variable.tf file of EC2

```
# to give variable value for volume az

variable "availability_zone" {
  description = "to add value for az"
  default     = "us-west-2c"
}

# to give variable value for volume size

variable "size" {
  description = "to add volume size"
  default     = 38
}

# to give value for instance type

variable "instance_type" {
  description = "to add instance type"
  default     = "t3.micro"
}

# to give value for device name

variable "device_name" {
  description = "to add device name"
  default     = "/dev/xvdbf"
}

# to add keypair in instance

variable "key_name" {
  description = "to add key name"
  default     = "terrakeypair"
}
```

Terraform output

- >> It is used to display the information about the resources created by terraform after execution.
- >> The moment terraform creates resources, output will show the details of the resources.

Uses of terraform output

1. To check important details like IPs, IDs, Hostname & DNS name.
2. To verify the creation of resource
3. Use values in CI/CD pipelines
4. To share the values between modules

Syntax of terraform output

```
output "resource_name" {  
    description = "description of output"  
    value = "resource_name.alias_name.id"  
}
```

How to get terraform output

- >> create the output.tf file
- >> Write code to get the specific output
- >> go for command terraform apply

List of outputs in EC2, S3 & VPC

EC2

instance_id
public_ip
private_ip
public_dns
security_groups
availability_zone
instance_type

S3

bucket_name
bucket_region
bucket_arn
bucket_domain_name
object_name

VPC

vpc_name
vpc_id
igw_id
subnet_name
route_table_ids
route_table_name
vpc_cidr
public_subnet_ids
private_subnet_ids
availability_zone

main.tf file of EC2

```
# to create the infrastructure
# to create instance
resource "aws_instance" "instancealias" {
  ami          = "ami-055a9df0c8c9f681c" # Amazon linux OS image ID
  instance_type = "t3.micro"
  key_name      = aws_key_pair.keypairalias.key_name      #syntax --> resourcename.aliasname.id

  tags = {
    Name = "terrainstance"
  }
}

# to create keypair
resource "aws_key_pair" "keypairalias" {
  key_name      = "terrakeypair"
  public_key    = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIL75AoFYjtAd
                  ZPB/i6JmjknFN/a9hd142zdMDlpeEVUktanvi@DESKTOP-7EKTOLQ"
}

# to create ebs volume
resource "aws_ebs_volume" "ebsalias" {
  availability_zone = "us-west-2b"
  size              = 45
  tags = {
    Name = "terravolume"
  }
}

# to attach the volume to instance
resource "aws_volume_attachment" "attachalias" {
  device_name = "/dev/xvdbf"
  volume_id   = aws_ebs_volume.ebsalias.id      # resourcename.aliasname.id --> syntax of
                                                    volume_id

  instance_id = aws_instance.instancealias.id   # resourcename.aliasname.id --> syntax of
                                                    instance_id
}
```

output.tf file of EC2

```
# to get the output of instance id
output "instance_id" {
  description = "output of instance id"
  value       = aws_instance.instancealias.id
}

# to get output of instance type
output "instance_type" {
  description = "output of instance type"
  value       = aws_instance.instancealias.instance_type
}

# to get output of key name
output "key_name" {
  description = "output of key name"
  value       = aws_key_pair.keypairalias.key_name
}

# to get output of availability zone
output "availability_zone" {
  description = "output of availability zone"
  value       = aws_ebs_volume.volumealias.availability_zone
}

# to get output of volume size
output "size" {
  description = "output of volume size"
  value       = aws_ebs_volume.volumealias.size
}

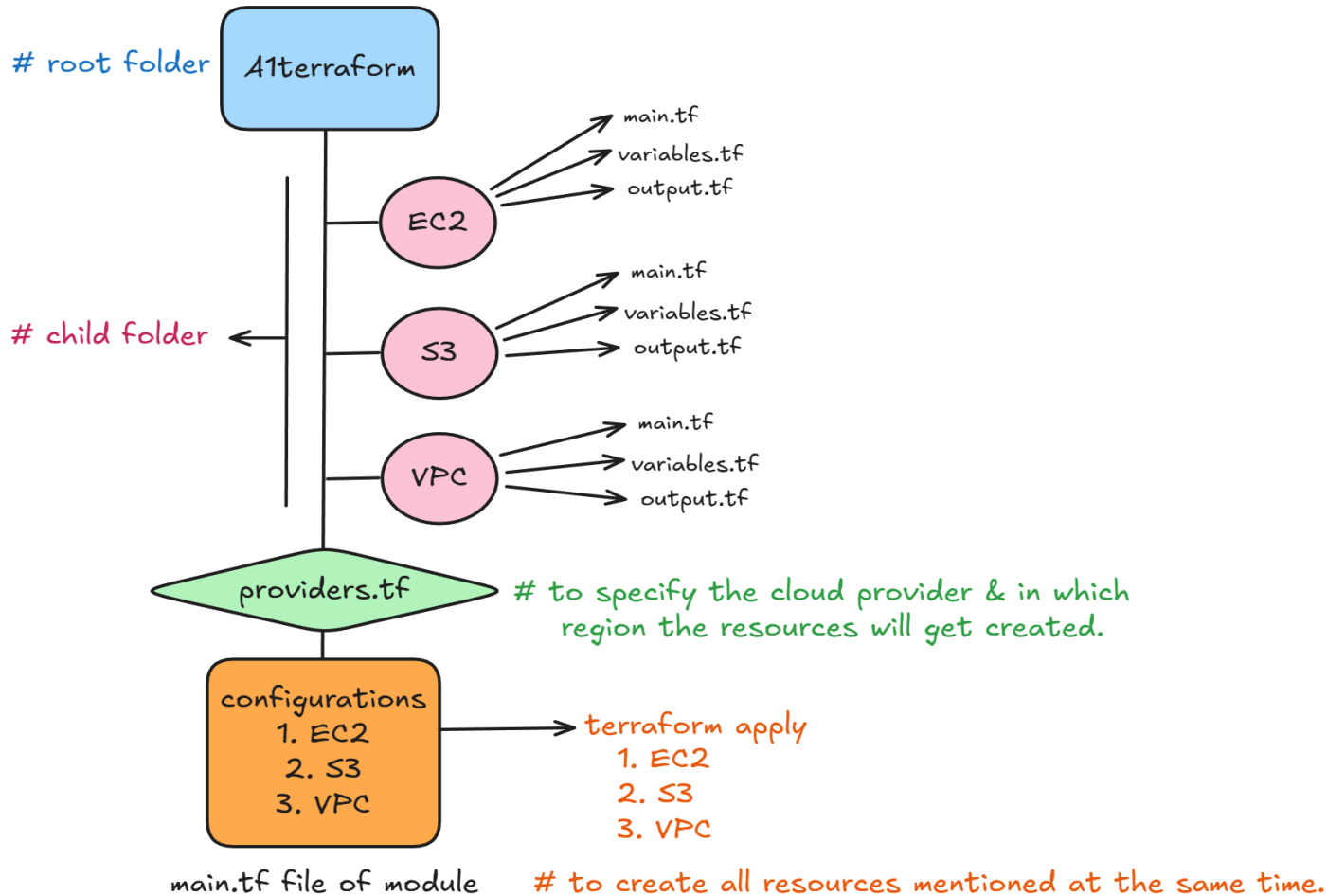
# to get output of public ip of instance
output "instance_public_ip" {
  description = "output of public ip"
  value       = aws_instance.instancealias.public_ip
}
```

Terraform modules

- >> It is the reusable folder that consist of infrastructure files to create the infrastructures/resources.
- >> It consists of terraform code to perform certain tasks.

Uses of terraform modules

1. We can reuse the code many times whenever needed
2. Organized code
3. Clean folder structure
4. Easy to maintain
5. It creates the infrastructures in faster way.



Syntax of module

```
module "module_name_1" {  
    source = "module_source_path"  
}
```

```
module "module_name_2" {  
    source = "module_source_path"  
}
```

main.tf file of module

to create the resources at same time

```
module "EC2" {  
    source = "./EC2"  
}
```

```
module "S3" {  
    source = "./S3"  
}
```

```
module "vpc" {  
    source = "./vpc"  
}
```